

Universidade de Coimbra
Licenciatura em Engenharia Informática
Compiladores

Nome: _____	Época Normal
Número de Aluno: _____	Duração: 123 min.

1. Linguagens regulares e análise lexical

Considere as categorias lexicais que se seguem:

- NAT** Sequência não vazia de dígitos entre 0 e 9.
- DEC** Sequência de um ou mais dígitos de 0 a 9 começada ou interrompida (mas não terminada) por um único ponto.
- HEX** Sequência de um ou mais dígitos de 0 a 9, iniciada por 0x; ou sequência de dígitos entre 0 e 9 e/ou letras de **a** até **f**, obrigatoriamente contendo pelo menos uma dessas letras e opcionalmente começada por 0x.

1.a)

Escreva uma expressão regular para cada uma das categorias lexicais apresentadas e apresente os autómatos finitos não-determinísticos que reconheçam as respetivas linguagens.

1.b)

Apresente o autómato finito determinístico de um analisador lexical que reconheça sequências de caracteres de cada uma das categorias lexicais e que faça a distinção entre estas, explicando as suas opções.

1.c)

Indique os símbolos e eventuais erros identificados durante a operação do analisador lexical sobre a seguinte linha de entrada:

"0xf0.20a00.00xf"

2. Análise sintática preditiva

Considere a linguagem caracterizada pela seguinte gramática livre de contexto:

- | | | | |
|-----|--------------|---|--------------------|
| (0) | <i>Start</i> | → | <i>L</i> \$ |
| (1) | <i>L</i> | → | [<i>I</i>] |
| (2) | <i>I</i> | → | id <i>N</i> |
| (3) | | | <i>L</i> <i>N</i> |
| (4) | <i>N</i> | → | , <i>I</i> |
| (5) | | | ε |

2.a)

Apresente uma sequência com exatamente 7 (sete) símbolos, sem contar com o fim de ficheiro (\$), que pertença à linguagem desta gramática, e construa uma árvore de derivação para essa mesma sequência.

2.b)

Apresente a tabela do analisador sintático LL(1) e mostre se a gramática é, ou não, LL(1), indicando todos os cálculos efetuados e justificando.

2.c)

Apresente a ordem dos nós na árvore sintática resultante da análise LL(1) para a seguinte sequência de entrada: [[id]]

3. Análise sintática ascendente e gramáticas livres de contexto

Considere a linguagem caracterizada pela gramática que se segue, que permite escrever fórmulas lógicas simples:

(0)	<i>Start</i>	$\rightarrow S \$$
(1)	<i>S</i>	$\rightarrow S F$
(2)		$\mid \varepsilon$
(3)	<i>F</i>	$\rightarrow \text{id} (P)$
(4)	<i>P</i>	$\rightarrow P \text{id}$
(5)		$\mid \varepsilon$

3.a)

Construa o autómato determinístico para controlar as ações de um analisador LR(1) para a gramática dada e prove se era ou não LR(1). Justifique as suas opções e os passos.

3.b)

Apresente um esquema de tradução (SDT) que permita construir a árvore de sintaxe abstrata durante a análise sintática ascendente. Considere as seguintes convenções (que poderá complementar), justificando:

```
/* cria um no de uma dada categoria,
com um simbolo lexical que podera ser nulo */
struct node *newnode(enum category category, char *token);

/* associa um no filho a um no pai */
void addchild(struct node *parent, struct node *child);
```

3.c)

Apresente a sequência de estados, o conteúdo da pilha e as ações realizadas por um analisador sintático LR(1) da gramática dada, tomando a expressão 'id (id)' como entrada.

4. Representação intermédia e geração de código

Considere o seguinte programa, escrito na linguagem C:

```

1 double sqrt(double x) {
2     double div = 2.0, err = 1e-6, try = x / div;
3     while (1) {
4         double next = try / div + x / try / div;
5         if (next - try < err && try - next < err)
6             break;
7         try = next;
8     }
9     return try;
10 }
```

4.a)

Apresente uma tradução do programa completo para código de três endereços.

4.b)

Determine o número mínimo de registos com os quais é possível avaliar a expressão da quarta linha do programa sem armazenar valores na memória (*spilling*). Justifique e apresente todos os cálculos necessários.

4.c)

Relativamente ao código final, qual deverá ser o conteúdo dos registos de ativação? Justifique as suas opções relativamente às alternativas possíveis.

Convenções C3E:

$x = y \text{ op } z$	<code>goto L</code>	<code>call p, n</code>	$x = y[i]$
$x = \text{uop } y$	<code>if x goto L</code>	<code>return</code>	$x[i] = y$
$x = y$	<code>if False x goto L</code>	$x = \text{call } p, n$	$x = \&y$
	<code>if x relop y goto L</code>	<code>return x</code>	$x = *y$
<code>param x</code>	$x = \text{param}[i]$		$*x = y$

Onde *op* denota um operador binário, *uop* denota um operador unário e *relop* denota um operador relacional.